

SOFTWARE TESTING

WAS IST SOFTWARE TESTING?

Software Testing ist der Prozess, bei dem Software auf Fehler überprüft wird, um sicherzustellen, dass sie den Anforderungen entspricht und korrekt funktioniert.

WARUM TESTEN?

born to **forced to write**
dilly dally **unit tests**



DHBW

Duale Hochschule
Baden-Württemberg

WARUM TESTEN?



Frühzeitiges Finden von Fehlern und Vermeidung von Bugs



Sicherstellung der Code-Qualität und Zuverlässigkeit



Schaffung von Vertrauen in die Funktionalität und Stabilität der Software



Lebendige Dokumentation des erwarteten Verhaltens



Reduzierung des Wartungsaufwands und langfristige Einsparung von Zeit und Kosten

ARTEN VON TESTS

STATISCHE TESTS

- 📄 Überprüfung des Codes ohne Ausführung
- 🔍 Code-Reviews, Linting, Statische Programmanalyse
- 🛡️ Frühe Fehlererkennung und Qualitätsverbesserung

DYNAMISCHE TESTS

- ▶️ Ausführung des Codes zur Überprüfung des Verhaltens
- 🐛 Manuelle Tests, Automatisierte Tests
- 🐛 Finden von Laufzeitfehlern und Validierung der Funktionalität

STATISCHE PROGRAMMANALYSE

Statische Programmanalyse ist eine Methode zur Überprüfung von Quellcode, ohne ihn auszuführen. Sie hilft, potenzielle Fehler, Sicherheitslücken und Code-Qualitätsprobleme frühzeitig zu erkennen.

- ⚙️ Überprüfung des Quellcodes ohne Ausführung
- 🔍 Erkennung von Syntaxfehlern, Stilproblemen und potenziellen Fehlern
- 🛡️ Verbesserung der Codequalität und -wartbarkeit
- 🚀 Automatisierte Überprüfung durch Tools wie Checkstyle, PMD, FindBugs, SonarQube

MANUELLE TESTS

Manuelle Tests sind Tests, die von Menschen durchgeführt werden, um die Funktionalität und Benutzerfreundlichkeit einer Software zu überprüfen. Sie sind besonders nützlich für explorative Tests und Usability-Tests.

EIGENSCHAFTEN

-  Durchführung durch Menschen (oft Endanwender)
-  Zeitaufwändig und fehleranfällig
-  Als Regressionstest sehr teuer

WANN SINNVOLL?

-  Bei schwer automatisierbaren Tests
-  UI-Tests (Texte, Layout, Design)
-  Exploratives Testing (unerwartete Fehler finden)

MANUELLE TESTS

VORTEILE ✓



Flexibel und anpassungsfähig



Gut für UI- und Usability-Tests



Kann unerwartete Fehler finden

NACHTEILE ✗



Zeitaufwändig und teuer



Fehleranfällig (menschliche Fehler)



Schwer reproduzierbar

AUTOMATISIERTE TESTS

Automatisierte Tests sind Tests, die mithilfe von Software-Tools und Skripten produktiven Code ausführen, um dessen Funktionalität und Leistung auf Korrektheit zu überprüfen.

EIGENSCHAFTEN

-  Durchführung durch Software-Tools
-  Schnell und wiederholbar
-  Kosteneffizient für Regressionstests

WANN SINNVOLL?

-  Wenn sich Anforderungen selten ändern und Tests wiederholt ausgeführt werden müssen
-  Wenn eine hohe Testabdeckung und schnelle Ausführung erforderlich sind
-  In agilen Entwicklungsprozessen und CI/CD-Pipelines

AUTOMATISIERTE TESTS

VORTEILE

 Schnell und wiederholbar

 Erleichtert Refactoring

 Hohe Testabdeckung möglich

NACHTEILE

 Initialer Aufwand zur Erstellung der Tests

 Wartungsaufwand bei Änderungen im Code

 Nicht geeignet für UI- und Usability-Tests

BLACK- VS. WHITE-BOX TEST

● BLACK-BOX TESTING



Testen ohne Kenntnis des internen Codes



Fokus auf Eingaben und erwartete

Ausgaben



Simuliert Benutzerverhalten

● WHITE-BOX TESTING



Testen mit Kenntnis des internen Codes

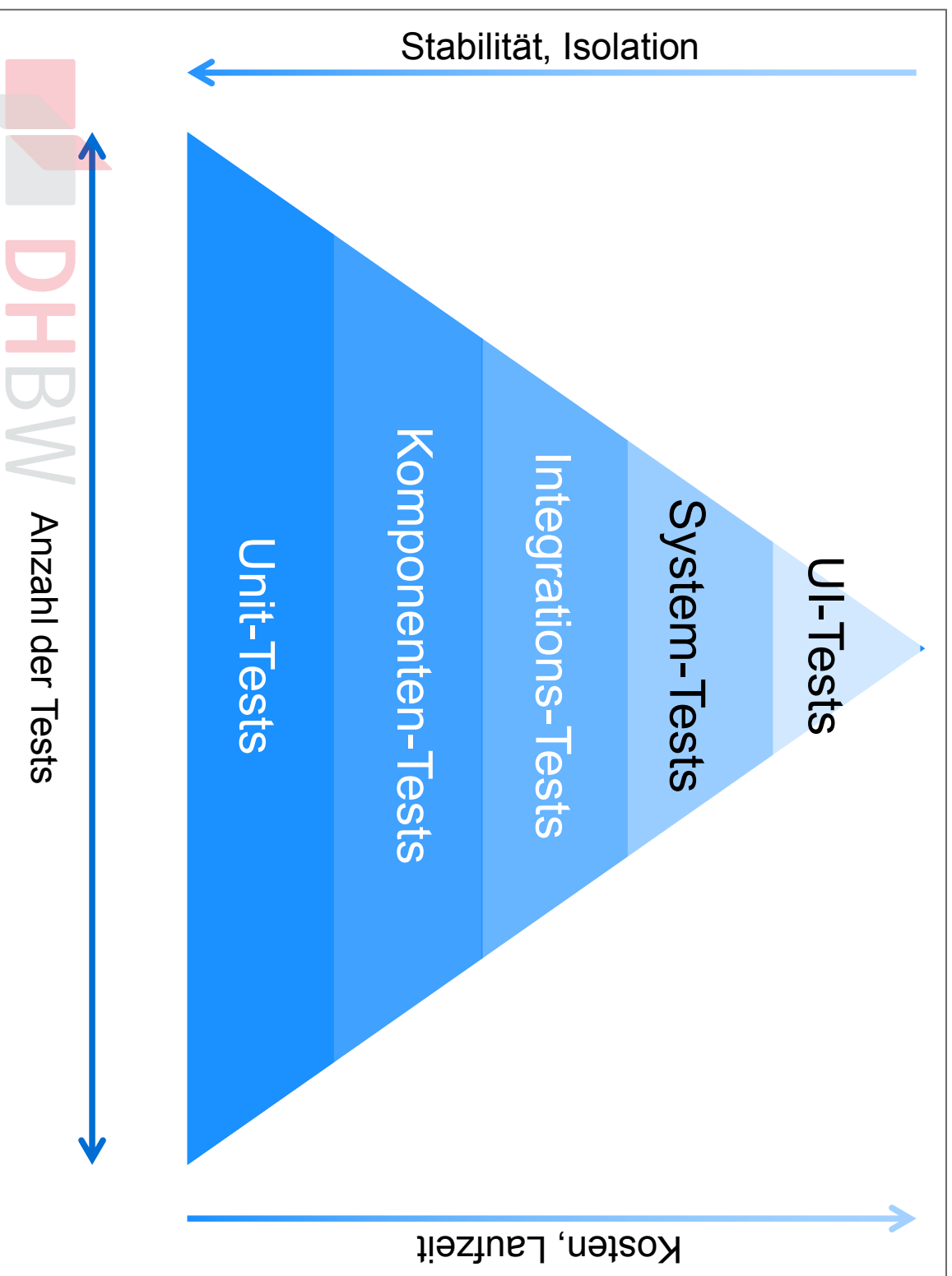


Fokus auf Code-Struktur und Logik



Ermöglicht gezielte Tests von Funktionen und Pfaden

TESTHIERARCHIE



Testpyramide

TESTHIERARCHIE

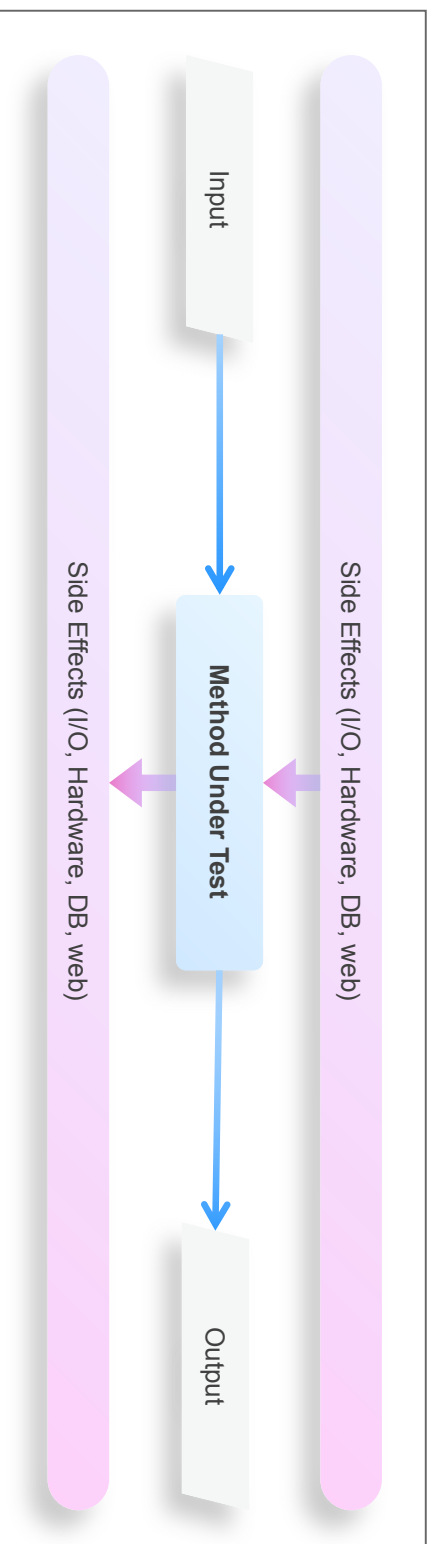
Testart	Beschreibung	Fokus
 Unit-Tests	Testen einzelner Funktionen oder Methoden	Isolation und Korrektheit einzelner Einheiten
 Komponenten-Tests	Testen einzelner Komponenten oder Module	Funktionalität und Zusammenspiel innerhalb einer Komponente
 Integration-Tests	Testen der Interaktion zwischen verschiedenen Komponenten	Zusammenarbeit und Datenfluss zwischen Komponenten
 System-Tests	Testen des gesamten Systems	Funktionalität, Leistung und Stabilität des Gesamtsystems
 UI-Tests	Testen der Benutzeroberfläche	Benutzerfreundlichkeit und Funktionalität der UI

WEITERE ARTEN VON TESTS

Testart	Beschreibung	Fokus
 Performance-Tests	Überprüfen die Geschwindigkeit und Skalierbarkeit der Software unter Last	Leistung und Effizienz unter verschiedenen Bedingungen
 Sicherheitstests	Identifizieren Sicherheitslücken und Schwachstellen in der Software	Schutz vor unbefugtem Zugriff und Datenverlust
 Usability-Tests	Bewerten die Benutzerfreundlichkeit und das Nutzererlebnis der Software	Benutzerfreundlichkeit und Zufriedenheit
 Regressionstests	Stellen sicher, dass neue Änderungen keine	Stabilität und Konsistenz nach Änderungen

Testart	Beschreibung	Fokus
	bestehenden Funktionen beeinträchtigen	
 End-to-End-Tests	Testen den gesamten Workflow der Anwendung von Anfang bis Ende	Vollständige Abdeckung der Anwendungsfunktionalität
 Smoke Tests	Überprüfen die grundlegende Funktionalität der Software nach einem Build oder einer Bereitstellung	Schnelle Validierung der grundlegenden Funktionen

TESTING IDEE



↓ Eingabe

Definieren der Eingabedaten für den Testfall.

↗ Erwartetes Ergebnis

Festlegen des erwarteten Ergebnisses basierend auf den Eingabedaten.

⚡ Side-Effects

Nebenwirkungen, welche die Ausführung der Funktion beeinflussen können und durch sie auftreten können.

Überprüfen des tatsächlichen Ergebnisses gegenüber dem erwarteten Ergebnis bei bekannter Eingabe und Minimierung von Side-Effects.

FIRST - PRINZIP

⚡ **Fast:** Tests sollten schnell ausgeführt werden, um häufiges Testen zu ermöglichen.

🧩 **Independent:** Tests sollten unabhängig voneinander sein, sich gegenseitig nicht beeinflussen und Seiteneffekte vermeiden.

🔄 **Repeatable:** Tests sollten unter verschiedenen Bedingungen reproduzierbar sein.

✅ **Self-Validating:** Tests sollten automatisch überprüfen, ob das Ergebnis korrekt ist.



Timely: Tests sollten früh im Entwicklungsprozess geschrieben werden, um Fehler frühzeitig zu erkennen.

WAS SOLLTEN TESTFÄLLE ABDECKEN?



Positive Testfälle

👉 Korrekte Werte



Negative Testfälle

👉 Inkorrekte Werte



Äquivalenzklassen

👉 Wertebereiche mit gleichem Verhalten



Grenzwertanalyse

👉 Werte an den Grenzen der Äquivalenzklassen und deren Nähe



Kombinationen von Eingabewerten

👉 z.B. durch all-pairs Testing

WAS SOLLTEN TESTFÄLLE ABDECKEN?

BEISPIEL

`boolean isPalindrome(String text) {...}`

<code>text = ""</code>	<code>true</code> / <code>Exception</code>	Spezifikation: Ist Leerer String ein Palindrom?
<code>text = "a"</code>	<code>true</code>	Ein einzelnes Zeichen ist ein Palindrom
<code>text = "abcd"</code>	<code>false</code>	Kein Palindrom
<code>text = "anna"</code>	<code>true</code>	Klassisches Palindrom
<code>text = "Anna"</code>	<code>true</code> / <code>false</code>	Spezifikation: Groß-/Kleinschreibung beachten?
<code>text = null</code>	<code>Exception</code>	Spezifikation: Wie mit null umgehen?

ARRANGE - ACT - ASSERT

Die "Arrange - Act - Assert" (AAA) Methode ist ein bewährtes Muster für das Schreiben von Tests, das hilft, Tests klar und verständlich zu strukturieren.

ARRANGE

ACT

ASSERT

 Vorbereiten der Testdaten  Ausführen der zu testenden Funktion.

 Beispiel: Erstellen eines Arrays von Zahlen für

 Beispiel: Aufrufen der Sortierfunktion mit dem

Sortieralgorithmus. vorbereiteten Array.  Fehler melden, wenn das Ergebnis falsch ist.

ARRANGE - ACT - ASSERT

Class Under Test (CUT)

```
1 class CustomMath {
2     private static double sum = 0;
3
4     public static int add(double[]
5         numbers) {
6         for(int i=0; i <
7             numbers.length; i++) {
8             sum += numbers[i];
9         }
10    }
11 }
```

Test Treiber Klasse

```
1 class TestDriver {
2     public static void main(String[] args)
3     {
4         // Arrange
5         double[] input = {1.0, 2.0, 3.0};
6         int expectedResult = 6.0;
7
8         // Act
9         int actualResult =
10            CustomMath.add(input);
11
12        // Assert
13        boolean testPassed = (actualResult
14            == expectedResult);
15        System.out.println(testPassed ?
16            "OK!" : "Failure!");
17    }
18 }
```

Konsolenausgabe:

● ● ●
> OK!



DHBW

Duale Hochschule
Baden-Württemberg

PROBLEM BEI DIESEM TEST

Class Under Test (CUT)

```
1 class CustomMath {
2     private static double sum = 0;
3
4     public static int add(double[]
5         numbers) {
6         for(int i=0; i <
7             numbers.length; i++) {
8             sum += numbers[i];
9         }
10    }
}
```

Test Treiber Klasse 2

```
1 class TestDriver2 {
2     public static void main(String[] args)
3     {
4         System.out.println("Test 1:");
5         double[] input1 = {1.0, 2.0, 3.0};
6         int expectedResult1 = 6.0;
7         CustomMath.add(input1);
8         boolean testPassed1 = (actualResult1
9             == expectedResult1);
10        System.out.println(testPassed1 ?
11            "OK!" : "Failure!");
12
13        System.out.println("Test 2:");
14        double[] input2 = {6.0};
15        int expectedResult2 = 6.0;
16        int actualResult2 =
17        CustomMath.add(input2);
18        boolean testPassed2 = (actualResult2
19            == expectedResult2);
20        System.out.println(testPassed2 ?
21            "OK!" : "Failure!");
22    }
}
```



```
> Test 1:
> OK!
> Test 2:
> Failure!
```



DHBW
Duale Hochschule
Baden-Württemberg

JUNIT

- ✍️ Erleichtert das Schreiben von Tests in Java
- ⚙️ Bietet Annotationen zur Kennzeichnung von Testmethoden
- ✅ Stellt *Assertions* zur Überprüfung von erwarteten Ergebnissen bereit
- 🔄 Lässt sich einfach in Entwicklungsumgebungen (IDEs) integrieren
- 🚀 Unterstützt Test Suites und Test Runner für die Organisation und Ausführung von Tests

JUNIT - BEISPIEL

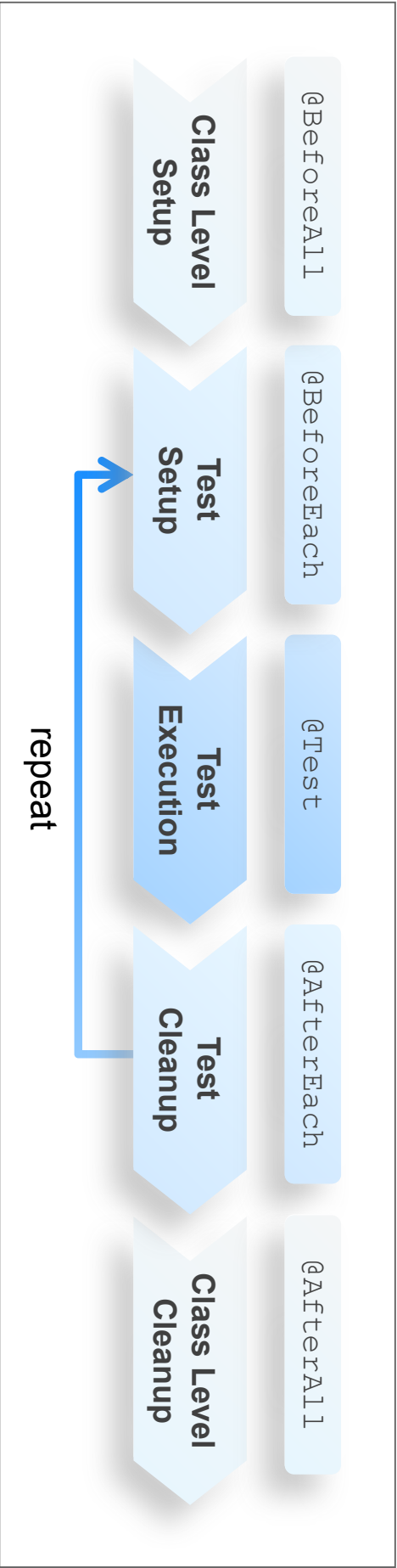
```
1 class CustomMathTest {
2
3     @Test
4     void testAdd() {
5         // Arrange
6         double[] input = {1.0, 2.0, 3.0};
7         int expectedResult = 6.0;
8
9         // Act
10        int actualResult = CustomMath.add(input);
11
12        // Assert
13        assertEquals(expectedResult, actualResult, "The sum should be 6.0");
14    }
15 }
```



DHBW

Duale Hochschule
Baden-Württemberg

JUNIT - TEST LIFECYCLE



Annotation	Zeitpunkt	Beschreibung
<code>@BeforeAll</code>	Einmalig vor allen Tests	Wird einmalig vor allen Testmethoden ausgeführt (Initialisierung).
<code>@BeforeEach</code>	Vor jedem Test	Wird vor jeder Testmethode ausgeführt (Initialisierung).
<code>@Test</code>	Testmethode	Kennzeichnet eine Methode als Testfall.
<code>@AfterEach</code>	Nach jedem Test	Wird nach jeder Testmethode ausgeführt (Aufräumen).

Annotation	Zeitpunkt	Beschreibung
@AfterAll	Einmalig nach allen Tests	Wird einmalig nach allen Testmethoden ausgeführt (Aufräumarbeiten).

JUNIT - ASSERTIONS

Assertion	Beschreibung
<code>assertEquals(expected, actual, message)</code>	Überprüft, ob zwei Werte gleich sind.
<code>assertNotEquals(unexpected, actual, message)</code>	Überprüft, ob zwei Werte ungleich sind.
<code>assertTrue(condition, message)</code>	Überprüft, ob eine Bedingung wahr ist.
<code>assertFalse(condition, message)</code>	Überprüft, ob eine Bedingung falsch ist.
<code>assertNotNull(object, message)</code>	Überprüft, ob ein Objekt null ist.
<code>assertNotNull(object, message)</code>	Überprüft, ob ein Objekt nicht null ist.

Assertion	Beschreibung
<code>assertThrows(expectedType, executable, message)</code>	Überprüft, ob eine bestimmte Exception geworfen wird.

JUNIT UMFASSENDES BEISPIEL

```
1 public class PalindromeChecker {
2     public boolean check(String text)
3     {
4         if(text == null) {
5             throw new
6             IllegalArgumentException("Cannot
7             check for palindrome on null");
8         }
9         if (text.isEmpty()) {
10            return false;
11        }
12        if(text.length() == 1) {
13            return true;
14        }
15        String cleanedText = text
16            .trim()
17            .toLowerCase();
18        String reversedText = new
19        StringBuilder(cleanedText)
20            .reverse()
21            .toString();
22        return
23        reversedText.equals(cleanedText);
24    }
25 }
```



```
1 public class PalindromeCheckerTest {
2
3     PalindromeChecker
4     palindromeChecker;
5
6     @BeforeEach setup() {
7         public
8         palindromeChecker = new
9         PalindromeChecker();
10    }
11
12    @Test @DisplayName("check should throw
13    IllegalArgumentException for null
14    input")
15    public void testCheck_NullInput()
16    {
17        @Throws(IllegalArgumentException
18        n.class, { }) ->
19        palindromeChecker.check(null);
20    }
21
22    @Test @DisplayName("check should return
23    false for empty string")
24    public void
25    testCheck_EmptyString() {
26        assertFalse(palindromeChecker.check(
27        ""));
28    }
29
30    @Test @DisplayName("check should return
31    true for single character string")
32 }
```

```

24 public void
25     testCheck_SingleCharacter() {
26         assertTrue(palindromeChecker.check("
27             a"));
28     }
29     @Test
30     @DisplayName("check should return
31         false for non-palindrome string")
32     public void
33         testCheck_NonPalindrome() {
34         assertFalse(palindromeChecker.check(
35             "hello"));
36     }
37     @Test
38     @DisplayName("check should return
39         true for palindrome string")
40     public void testCheck_Palindrome()
41     {
42         assertTrue(palindromeChecker.check("
43             madam"));
44     }
45     @Test
46     @DisplayName("check should return
47         true for palindrome with mixed
48         trailing spaces")
49     public void
50         testCheck_Palindrome_LeadingTrailing

```



```
49 Spaces() {  
    assertTrue(palindromeChecker.check(  
50         madam  
51     });  
}
```



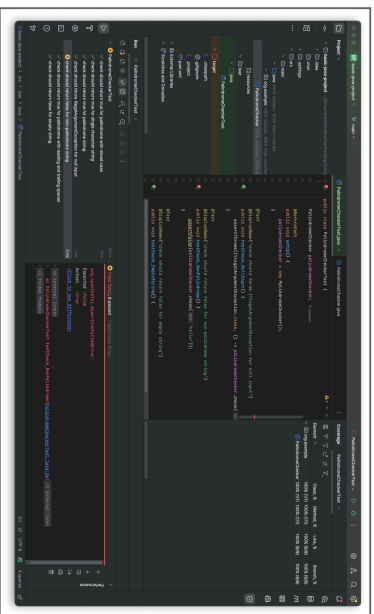
DHBW
Duale Hochschule
Baden-Württemberg

TEST IN DER IDE

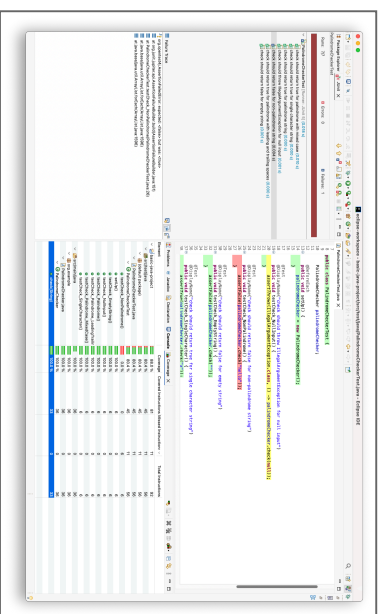


TEST IN DER IDE

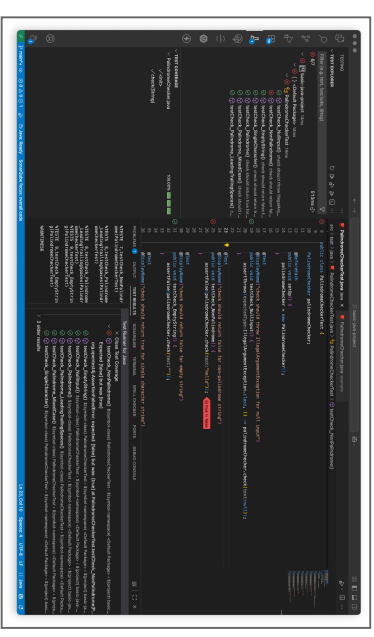
INTELLIJ IDEA



ECLIPSE IDE

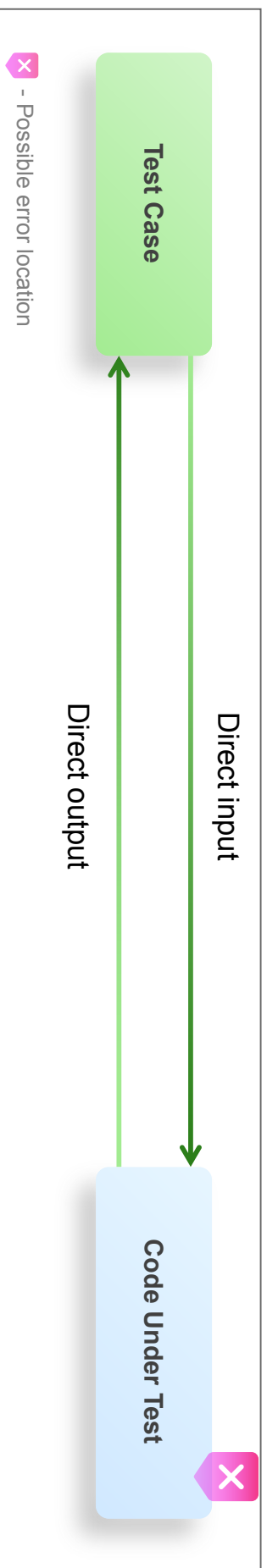


VISUAL STUDIO CODE



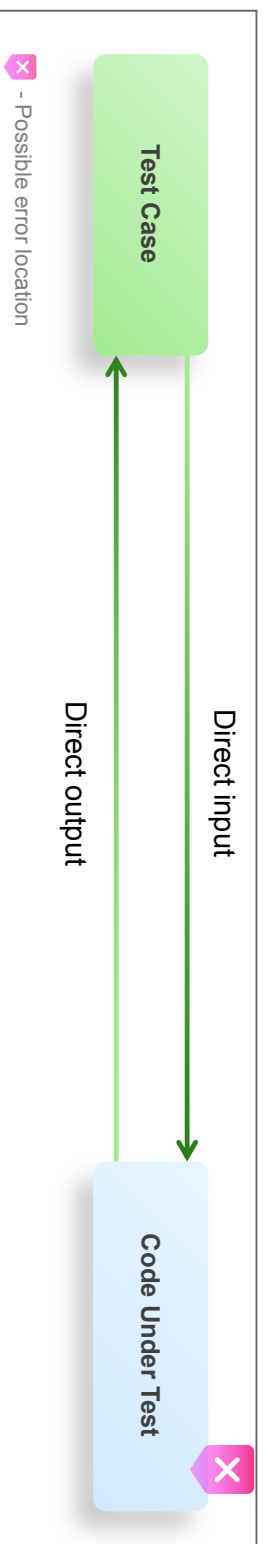
TEST ISOLATION

TEST-CODE OHNE ABHÄNGIGKEITEN



TEST ISOLATION

TEST-CODE OHNE ABHÄNGIGKEITEN



🌈 Im Idealfall hat der zu testende Code hat keine Abhängigkeiten

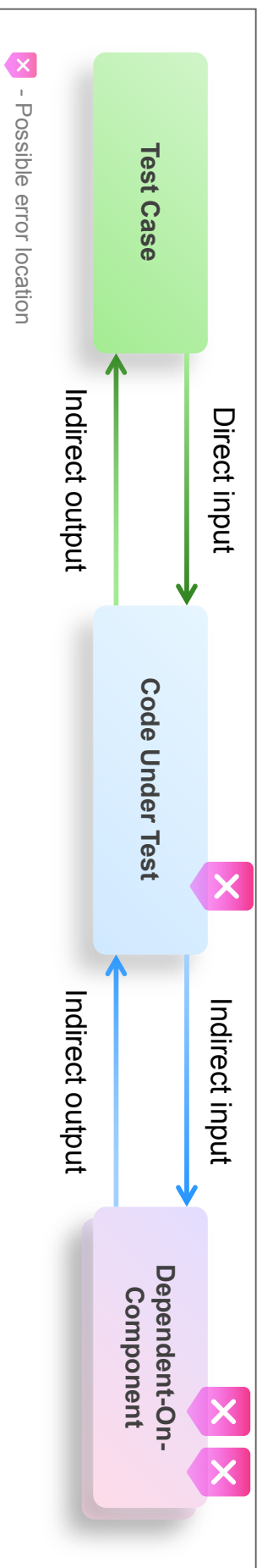
⚡ Tests sind schnell ausführbar

🔄 Tests sind unabhängig und wiederholbar

🎯 Fehlerquellen können nur im code-under-test liegen

TEST ISOLATION

TEST-CODE MIT ABHÄNGIGKEITEN



TEST ISOLATION

TEST-CODE MIT ABHÄNGIGKEITEN



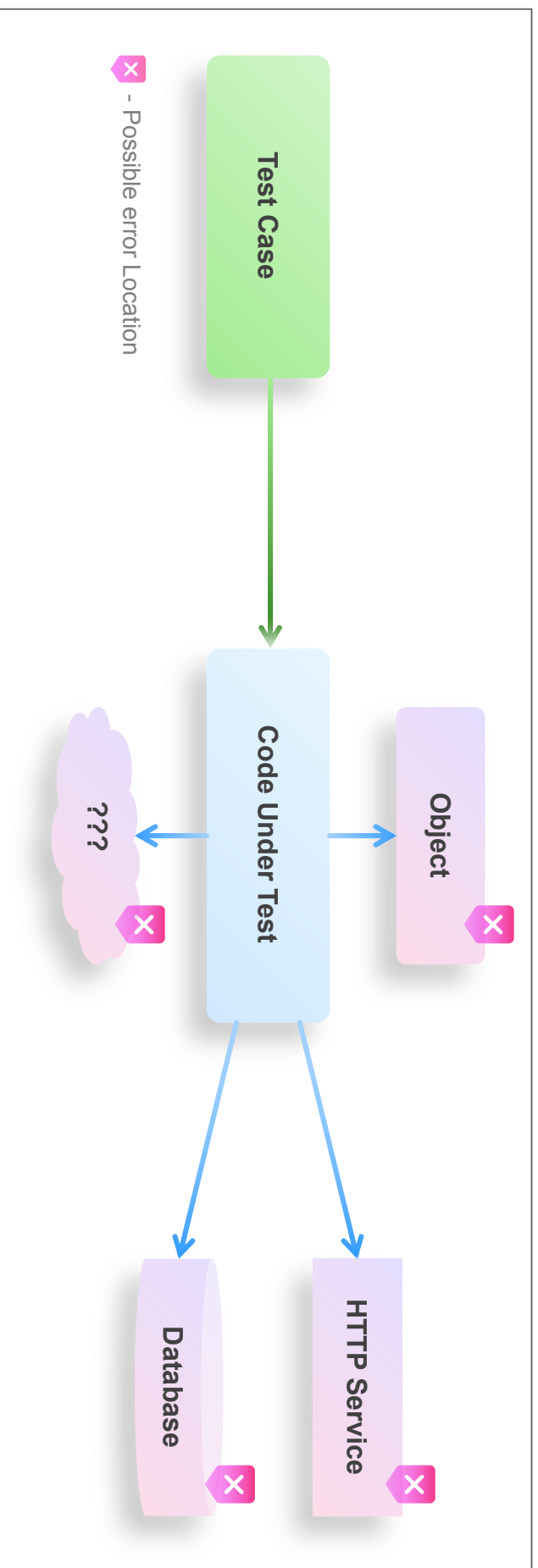
⚠️ In der Praxis hat der zu testende Code oft Abhängigkeiten

🐢 Tests können langsam und unzuverlässig werden

❗ Fehlerquellen können im code-under-test oder in Abhängigkeiten liegen

TEST ISOLATION

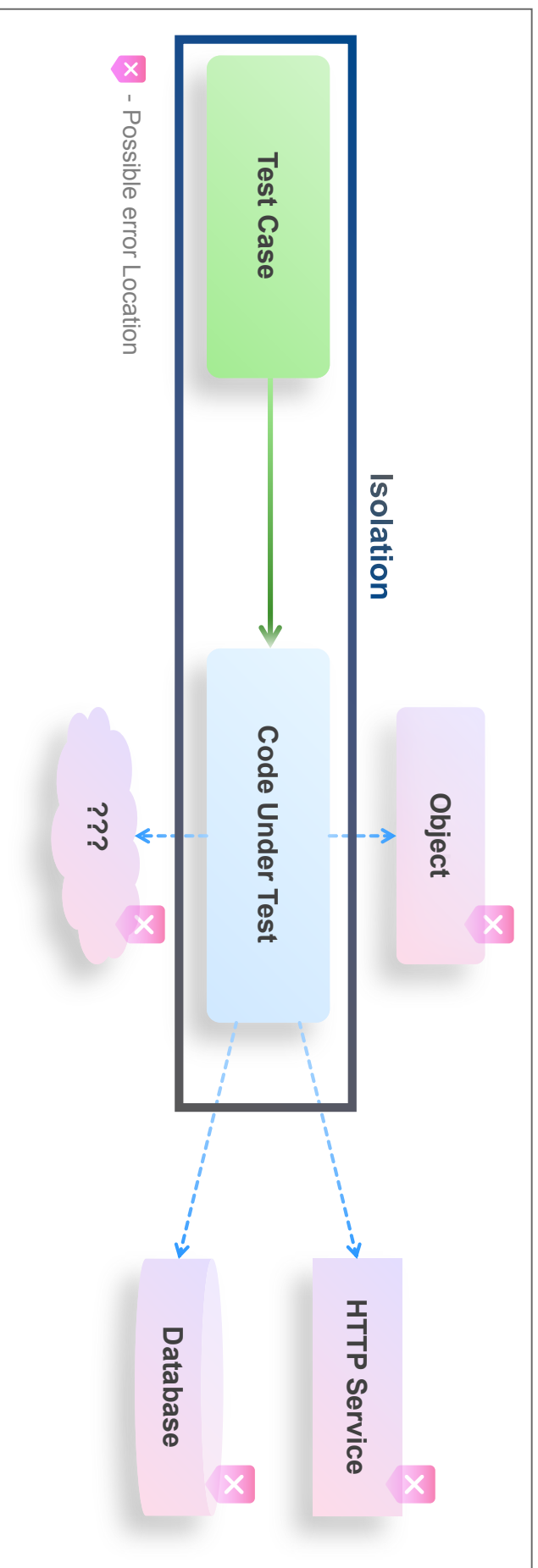
TEST-CODE MIT ABHÄNGIGKEITEN



👉 Wir wollen die Abhängigkeiten isolieren, um die Tests schnell, unabhängig und wiederholbar zu machen.

TEST ISOLATION

TEST-CODE MIT ABHÄNGIGKEITEN

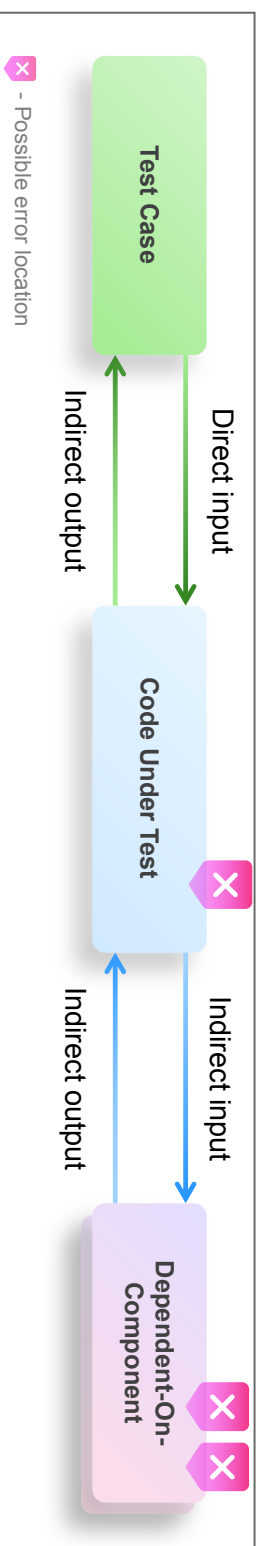


👉 Wir wollen die Abhängigkeiten isolieren, um die Tests schnell, unabhängig und wiederholbar zu machen.

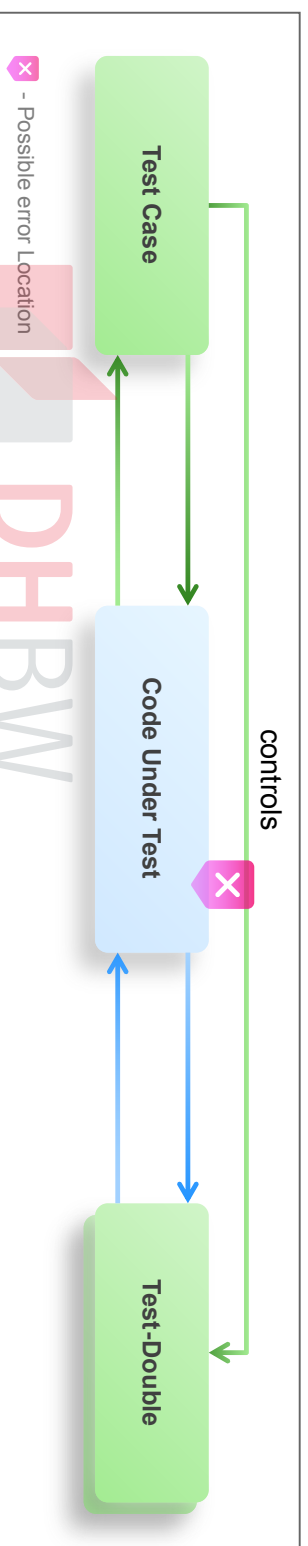
TEST ISOLATION

TEST DOUBLES

Vorher:

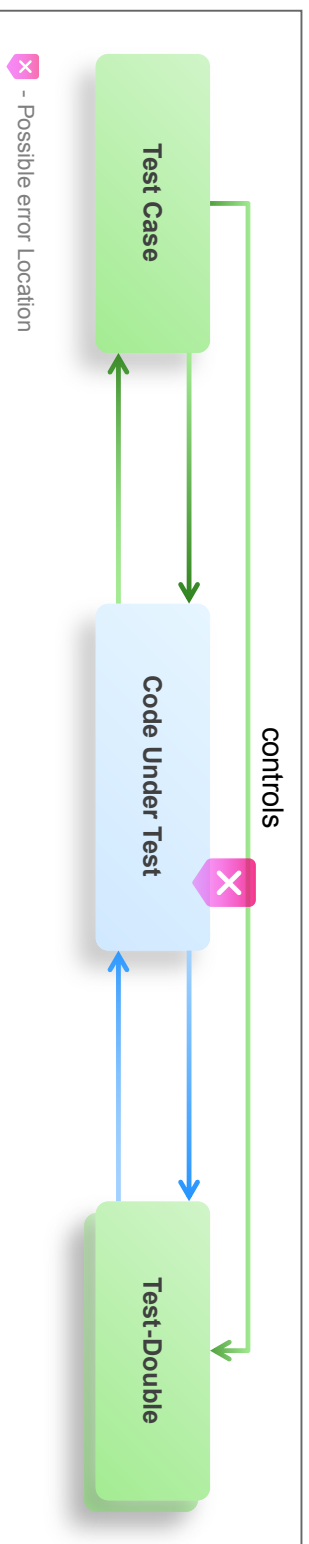


Nachher:



TEST ISOLATION

TEST DOUBLES



Test Doubles sind Objekte, die die Abhängigkeiten des zu testenden Codes ersetzen



Sie ermöglichen es dem Test, das Verhalten der Abhängigkeiten zu kontrollieren und zu simulieren



Dadurch können Tests isoliert und reproduzierbar durchgeführt werden

TEST DOUBLES

ARTEN VON TEST DOUBLES

 Beschreibung	 Ziel
Dummy Objekte, die als Platzhalter übergeben werden, aber über kein Verhalten verfügen.	Erfüllung von Abhängigkeiten, ohne tatsächliches Verhalten zu simulieren.
Fake Objekte mit einer minimalen, dem Testfall genügenden Implementierung.	Ersatz von komplexen Abhängigkeiten durch einfachere Alternativen.
Stub Liefert vordefinierte Antworten auf bestimmte Aufrufe.	Kontrolle der indirekten Eingaben des zu testenden Codes.
Spy Protokolliert und überprüft ggf., ob bestimmte Interaktionen stattgefunden haben.	Überprüfung der indirekten Ausgaben des zu testenden Codes.



Beschreibung

Ziel

	Objekte, die vordefinierte Antworten liefern und zusätzlich das erwartete Verhalten überprüfen.	Umfassende Kontrolle und Überprüfung der Interaktionen mit Abhängigkeiten.
Mock		

COVERAGE

Woher weiß ich, dass ich genug getestet habe?

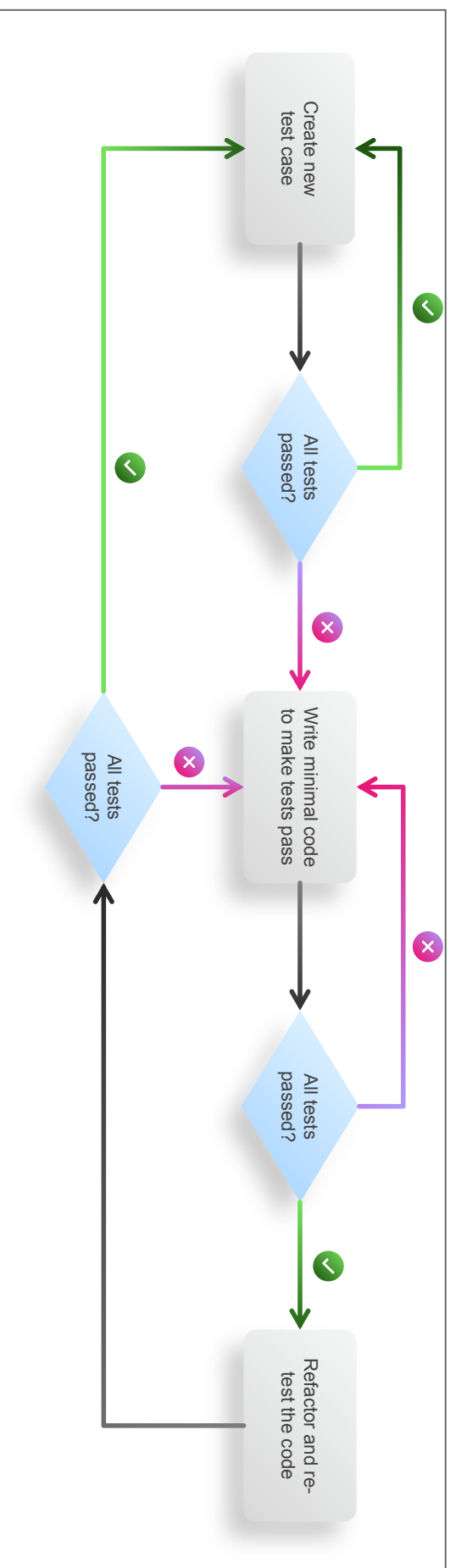
COVERAGE

-  Code Coverage misst den Anteil des Codes, der durch Tests abgedeckt ist
-  Höhere Coverage bedeutet nicht automatisch bessere Tests
-  ⚠️ Wichtiger ist die Qualität der Tests und die Abdeckung verschiedener Szenarien
-  🎯 Ziel ist es, kritische Pfade und Randfälle abzudecken
-  100% Coverage selten erreichbar
-  🏆 Eine Coverage von 80-90% ist ein guter Richtwert, aber wichtiger ist die Qualität der Tests

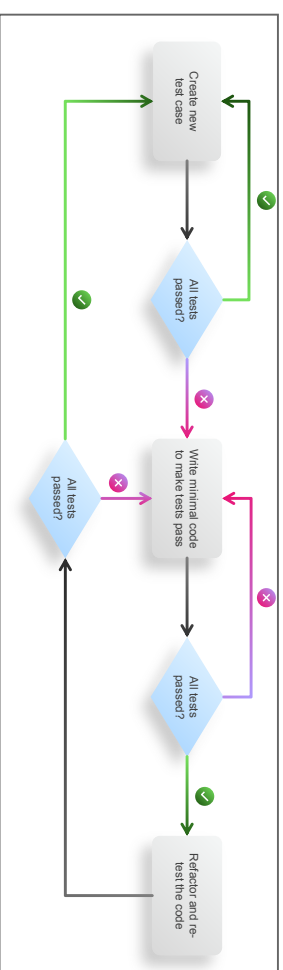
COVERAGE ARTEN

Art	Beschreibung
 Line Coverage	Misst den Prozentsatz der ausgeführten Codezeilen.
 Statement Coverage	Misst den Prozentsatz der ausgeführten Anweisungen.
 Branch Coverage	Misst den Prozentsatz der ausgeführten Verzweigungen (if, switch).
 Path Coverage	Misst den Prozentsatz der durchlaufenen Pfade im Kontrollflussgraphen.

TEST-DRIVEN DEVELOPMENT



TEST-DRIVEN DEVELOPMENT



VORTEILE ✓

- 👤 Zwingt Entwickler, sich mit allen möglichen Programmsituationen auseinanderzusetzen
- 🔪 Garantiert Existenz von Tests
- 🐛 Verschleppung von Fehlern wird verhindert
- ✓ Es werden genau die Anforderungen erfüllt – und nicht mehr!
- 🚀 Es wird genau nur soviel Produktivcode geschrieben wie benötigt
- ✨ Das Interface-Design wird in der Regel sehr sauber

NACHTEILE ✗

- 😓 Akzeptanz von Entwicklern nicht immer gegeben
- 🔧 Konsequente Umsetzung notwendig
- ⚠️ Wenig geeignet für Prototyping

Speaker notes